Gradient-Free Low-Rank Adaptation of Large Language Models for Few-Shot Learning

DOMAIN: Artificial Intelligence, Large Language Models, Natural Language Processing

TEAM - 37

PROJECT SUPERVISOR:

Dr. B. Thanasekhar

PROFESSOR

Department of Computer Technology
MIT Campus, Chennai - 600044

TEAM MEMBERS:

Tanya M Cherry (2022503529)

Priya Dharshini C (2022503501)

Mukeshraj S (2022503309)

PROBLEM STATEMENT

- Full fine-tuning of LLMs is computationally expensive and memory-intensive.
- In few-shot settings, updating all parameters often causes overfitting and instability.
- Parameter-efficient methods like prompt tuning, adapters, LoRA reduce trainable parameters but still rely on backpropagation, limiting scalability.
- Existing gradient-free methods focus mainly on prompt tuning and suffer from slow convergence.
- There is a need for a stable, efficient, and gradient-free fine-tuning approach for few-shot learning.

OBJECTIVES

- To design a parameter-efficient fine-tuning framework for large language models that avoids gradient-based optimization.
- To integrate low-rank adaptation modules into the self-attention layers of a pretrained language model.
- To propose and apply new gradient-free low-rank adaptation methods, namely C-LoRA and F-LoRA, for optimizing low-rank parameters in few-shot learning scenarios.
- To evaluate the proposed approach on benchmark datasets and compare its performance with existing gradient-based and gradient-free fine-tuning methods.
- To analyze the efficiency of the proposed method in terms of performance, memory usage, and convergence behavior.

LITERATURE SURVEY

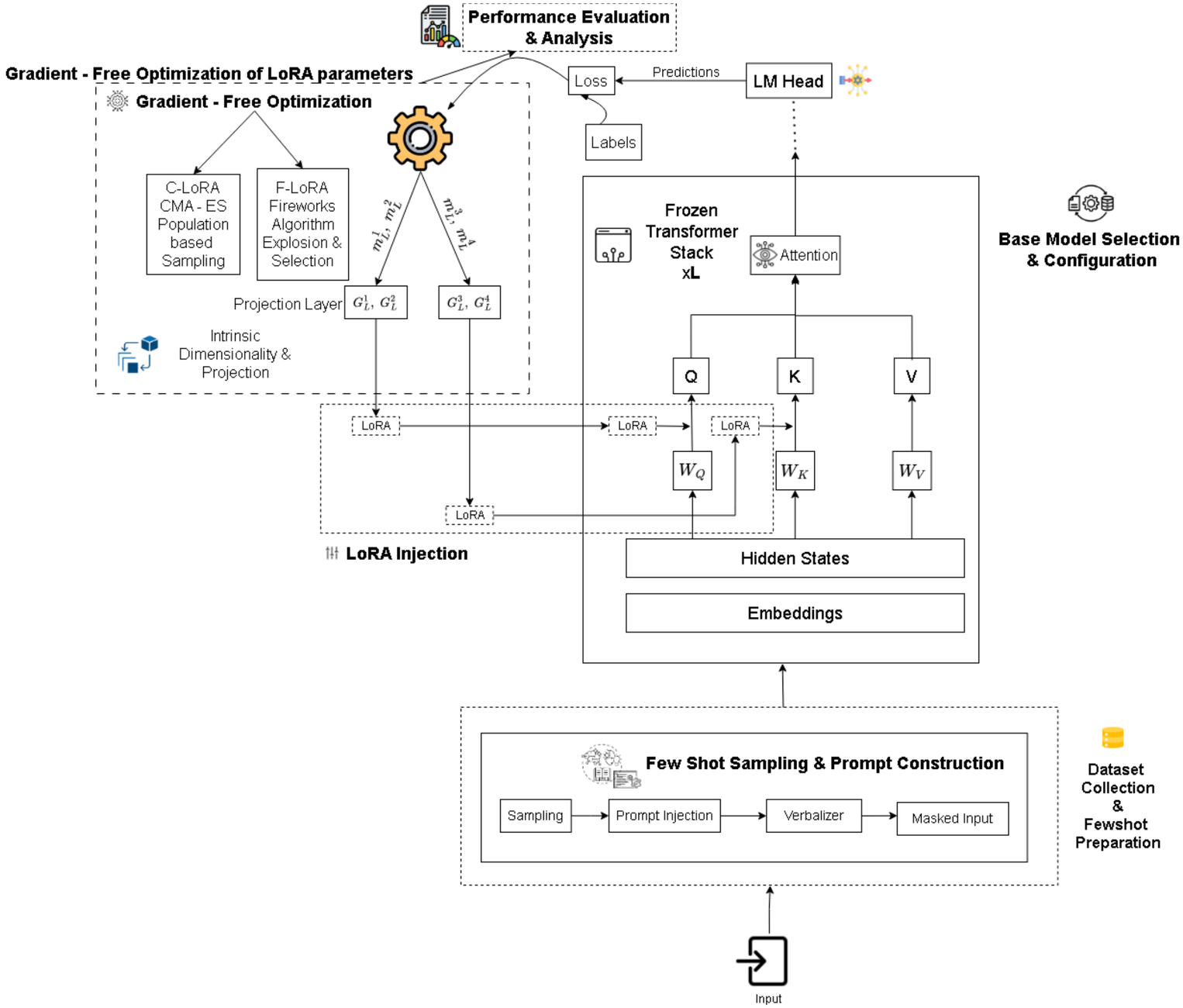
S.NO	NAME OF THE JOURNAL	TITLE	LIMITATIONS	METHODOLOGY USED
[1]	IEEE Transactions on Audio, Speech and Language Processing 2025	HybridMoE: LoRA-Based LLMs Fine-Tune With Hybrid Mixture of Experts	◦ Although HybridMoE reduces trainable parameters, it still relies on gradient-based optimization and backpropagation, which incurs non-trivial computational and memory overhead during fine-tuning. ◦ The training process involves complex routing and expert coordination, which can increase optimization instability, especially in low-resource or few-shot settings.	◦ Combines LoRA-based parameter-efficient fine-tuning with a hybrid Mixture-of-Experts (MoE) architecture for multitask LLM adaptation. ◦ Introduces Multi-A MoE and Multi-B MoE to capture task diversity and task similarity by selectively sharing LoRA decomposition matrices.

S.NO	NAME OF THE JOURNAL	TITLE	LIMITATIONS	METHODOLOGY USED
[2]	IEEE Internet of Things Journal 2025	SecLoRA: Efficient Privacy-Preserving LLM Tuning	◦ Focuses mainly on LoRA-based tuning, limiting applicability to other PEFT methods. ◦ Adds cryptographic and communication overhead, making deployment harder for resource-constrained clients.	◦ Proposes local LoRA pre-optimization to accelerate convergence before secure cloud fine-tuning. ◦ Designs SecLoRA using IPFE and matrix permutation to protect data and LoRA parameters during outsourced training.
[3]	IEEE Control Systems Letters 2021	A Derivative-Free Optimization Method With Application to Functions With Exploding and Vanishing Gradients	◦ Performance is sensitive to hyperparameters like agent count, gain, and step size. ◦ Convergence is guaranteed only outside small neighborhoods of local minima.	◦ Uses a SUSD-based derivative-free optimization algorithm. ◦ Uses exponential objective mapping to manage exploding and vanishing gradients.

S.NO	NAME OF THE JOURNAL	TITLE	LIMITATIONS	METHODOLOGY USED
[4]	IEEE Transactions on Cybernetics 2022	A Reference Vector-Based Simplified Covariance Matrix Adaptation Evolution Strategy for Constrained Global Optimization	◦ Efficiency depends on proper tuning of reference vectors. ◦ Increased computational overhead due to ranking, repair, and covariance updates in high-dimensional problems.	◦ Converts constrained problems into a bi-objective form using simplified CMA-ES. ◦ Applies reference vector ranking with repair and restart for better convergence.
[5]	IEEE Access 2020	Optimal Performance and Application for Firework Algorithm Using a Novel Chaotic Approach	◦ Performance depends on proper selection of chaotic maps. ◦ Chaotic operations increase computational overhead and may not always outperform adaptive FWA variants on all functions.	◦ Integrates hybrid chaotic maps into the Firework Algorithm to enhance exploration and convergence. ◦ Uses chaos-based initialization and adaptive disturbance to avoid local optima.

S.NO	NAME OF THE JOURNAL	TITLE	LIMITATIONS	METHODOLOGY USED
[6]	IEEE Transactions on Evolutionary Computation, Vol. 24, No. 1, February 2020	Toward a Matrix-Free Covariance Matrix Adaptation Evolution Strategy	◦ Lacks step-size multiplier adaptation found in standard CMA-ES. ◦ Requires storing historical data, increasing memory overhead.	◦ Proposes a matrix-free CMA-ES using differential mutation and Gaussian corrections. ◦ Introduces DES to model covariance adaptation without explicit matrices.
[7]	IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI), Vol. 47, No. 11, 2025	Toward Few-Shot Learning in the Open World: A Review and Beyond	◦ Focuses on summarizing existing methods without proposing a new framework. ◦ Relies on reported results, which may vary across datasets and settings.	◦ Provides a comprehensive survey of Open World Few-Shot Learning (OWFSL). ◦ Categorizes OWFSL into varying instances, classes, and distributions.

ARCHITECTURE DIAGRAM OF THE PROPOSED SYSTEM



PROJECT MODULES & DESCRIPTIONS

1. Dataset Collection and Few-Shot Preparation

Input:

Raw NLP datasets.

Output:

True few-shot datasets with prompt-based inputs and verbalizers.

Description:

- Curates benchmark NLP datasets (SST-2, Yelp, AG's News, DBPedia) using the Hugging Face Datasets library for few-shot experiments.
- A true few-shot strategy (K = 16) is applied per class to create balanced training and validation splits for fair evaluation.
- Tasks are reformulated using prompt templates and verbalizers

PROJECT MODULES & DESCRIPTIONS

2. Base Model Selection and Configuration

Input:

Prompt-formatted few-shot datasets and pretrained language model.

Output:

GPU-ready frozen large language model with validated forward-pass execution.

Description:

- Selects suitable pretrained model (RoBERTa-Base) and loads weights using the Hugging Face Transformers framework.
- All backbone parameters are frozen, and the self-attention components (Q, K, V) are identified as targets for later adaptation.

PROJECT MODULES & DESCRIPTIONS

3. Low-Rank Adaptation (LoRA) Injection

Input:

Frozen pretrained language model with identified self-attention layers.

Output:

LoRA-augmented language model with a small number of trainable parameters.

Description:

- Integrates Low-Rank Adaptation (LoRA) into the frozen language model using low-rank matrices with r = 8.
- LoRA is applied only to the Query (Q) and Key (K) projections of each Transformer layer, while the backbone weights remain frozen.
- LoRA is applied independently across layers.

PROJECT MODULES & DESCRIPTIONS

4. Gradient-Free Optimization of LoRA Parameters

Input:

LoRA-augmented language model and few-shot training data.

Output:

Optimized LoRA parameters obtained without gradient computation.

Description:

- Optimizes LoRA parameters without backpropagation, searching in a low-dimensional intrinsic space to reduce optimization complexity.
- Intrinsic latent vectors are projected to LoRA parameters using projection matrices initialized from hidden-state statistics for stable mapping.
- CMA-ES (C-LoRA) and Fireworks Algorithm (F-LoRA) are applied using forward-pass evaluations only, with layer-wise optimization and fixed budgets for stable convergence.

PSEUDOCODE OF PROPOSED ALGORITHM

Algorithm 1: DFOs for Low - Rank Adaptation

Require: L-layer language model f ,
Budget of API calls: $\mathcal{B}$,
Optimizers: $\{O_i\}_i^L = 1$,
Loss function $\mathcal{L}$

Hidden Variable: $m_L^i, i = 1, 2, 3, 4$
Random projections: $G_L^i, i = 1, 2, 3, 4$
Low-rank modules of each layer: A and B
repeat
  for each hidden layer **do**
    Evaluate: $loss = L\left(f\left(G_L^i m_L^i\right)\right)$
    Update m_L^i by DFOs: $O_L^i\left(m_{L,}^i loss\right)$
    Update the Low-rank modules A and B through $G_L^i m_L^i$ while keep G_L^i frozen
  end for
until $\mathcal{B}$ / L times

PROJECT MODULES & DESCRIPTIONS

5. Experimental Evaluation and Performance Analysis

Input:

Optimized models (Baseline, LoRA, C-LoRA, F-LoRA) and held-out test datasets.

Output:

Quantitative and qualitative evaluation results with comparative analysis.

Description:

- Evaluates few-shot performance using Accuracy and F1-score for classification.
- The performance of Baseline, gradient-based LoRA, C-LoRA, and F-LoRA is systematically compared across benchmark datasets.
- GPU memory and training time are monitored to analyze efficiency and convergence, with results shown using graphs and tables.

IMPLEMENTATION RESULTS – Module 1

SST-2 dataset processed in a few-shot setting, illustrating efficient data loading, filtering, and few-shot split generation.

```
============================================================
PROCESSING SST-2
============================================================

============================================================
Processing sst2 with 16-shot setting
============================================================
Loading dataset...
README.md: 5.27k/? [00:00<00:00, 163kB/s]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
data/train-00000-of-00001.parquet: 100% 3.11M/3.11M [00:01<00:00, 2.38MB/s]
data/validation-00000-of-00001.parquet: 100% 72.8k/72.8k [00:00<00:00, 142kB/s]
data/test-00000-of-00001.parquet: 100% 148k/148k [00:00<00:00, 15.9kB/s]
Generating train split: 100% 67349/67349 [00:00<00:00, 603331.41 examples/s]
Generating validation split: 100% 872/872 [00:00<00:00, 35707.01 examples/s]
Generating test split: 100% 1821/1821 [00:00<00:00, 60746.40 examples/s]
Using validation set as test set (original test set has no labels)
Valid labels: {0, 1}
Filter: 100% 67349/67349 [00:00<00:00, 158832.36 examples/s]
Filter: 100% 872/872 [00:00<00:00, 33632.19 examples/s]
Original train size (after filtering): 67349
Original test size (after filtering): 872
 Creating 16-shot splits...
 Few-shot train size: 32
 Few-shot dev size: 32

 Applying patterns and verbalizers...
Map: 100% 32/32 [00:00<00:00, 1339.89 examples/s]
Map: 100% 32/32 [00:00<00:00, 1285.00 examples/s]
Map: 100% 872/872 [00:00<00:00, 7410.19 examples/s]

 --- Sample from training set ---
 Original text: exactly how genteel and unsurprising the execution turns out to be ...
 MLM text: exactly how genteel and unsurprising the execution turns out to be  It was <mask>....
 Label: 0
 Verbalizer: bad
```

Sample outputs showing label–verbalizer mapping and masked-text inputs used for few-shot classification across datasets.

	label	verbalizer	text_preview	mlm_text_preview
0	0	bad	exactly how genteel and unsurprising the execution turns out to be ...	exactly how genteel and unsurprising the execution turns out to be It was <mask>....
1	0	bad	an all-time low for kevin costner ...	an all-time low for kevin costner It was <mask>....
2	0	bad	several cliched movie structures : ...	several cliched movie structures : It was <mask>....
3	0	bad	america 's knee-jerk moral sanctimony ...	america 's knee-jerk moral sanctimony It was <mask>....
4	0	bad	a dumb comedy ...	a dumb comedy It was <mask>....
5	0	bad	the plan to make enough into ` an inspiring tale of survival wrapped in the heart-pounding suspense ...	the plan to make enough into ` an inspiring tale of survival wrapped in the heart-pounding suspense of a stylish psychological thriller ' has flopped ...
6	0	bad	proficient , dull ...	proficient , dull It was <mask>....
7	0	bad	chitchat that only self-aware neurotics engage in ...	chitchat that only self-aware neurotics engage in It was <mask>....
8	0	bad	tries to cram too many ingredients into one small pot	tries to cram too many ingredients into one small pot . It was <mask>....
9	0	bad	paper-thin characterizations ...	paper-thin characterizations It was <mask>....
10	0	bad	this overproduced piece ...	this overproduced piece It was <mask>....

	label	verbalizer	text_preview	mlm_text_preview
0	0	World	Aid sought for storm damage Officials yesterday sought international help to rebuild villages ravage...	<mask> News Aid sought for storm damage Officials yesterday sought international help to rebuild villages ravaged by back-to-back storms that left 566...
1	0	World	Basque group sets off 7 explosions in Spain MADRID The Basque separatist group ETA struck Monday for...	<mask> News Basque group sets off 7 explosions in Spain MADRID The Basque separatist group ETA struck Monday for the second time in days, exploding se...
2	0	World	France #39;s Ban on Head Scarves Put to Test PARIS Sept. 2, 2004 - France #39;s bold effort to reign...	<mask> News France #39;s Ban on Head Scarves Put to Test PARIS Sept. 2, 2004 - France #39;s bold effort to reign in Muslim fundamentalism was put to t...
3	0	World	EU backing for Romania elections The European Commission says the elections in Romania were conducte...	<mask> News EU backing for Romania elections The European Commission says the elections in Romania were conducted in an orderly manner and will not af...
4	0	World	Six Shot Dead in New Northeast India Violence (Reuters) Reuters - Suspected tribal rebels stormed\a ...	<mask> News Six Shot Dead in New Northeast India Violence (Reuters) Reuters - Suspected tribal rebels stormed\a remote village in India's northeast on...
5	0	World	Mob rule takes a hit as Italy fights back MAFIA investigators christened him The Ambassador. After a...	<mask> News Mob rule takes a hit as Italy fights back MAFIA investigators christened him The Ambassador. After a seven-year trial in the bunker of a P...

IMPLEMENTATION RESULTS – Module 3

LoRA injected into RoBERTa by freezing base parameters and enabling training only on low-rank adapters in attention layers.

```
================================================================================
INJECTING LoRA INTO RoBERTa MODEL
================================================================================

Freezing all original model parameters

Injecting LoRA (rank=8, alpha=16.0) into attention layers
   Target modules: ['query', 'key']

Enabling gradients for LoRA parameters only...

LoRA Injection Statistics:
   Total layers: 12
   LoRA-injected layers: 12
   Original parameters: 124,645,632
   LoRA parameters: 294,912
   Trainable parameters: 294,912
   Trainable %: 0.24%

✓ LoRA injection complete!
```

Verifies that only the LoRA low-rank matrices (A and B) in each attention layer are set as trainable parameters, while all original model weights remain frozen.

```
======================================================================
VERIFYING TRAINABLE PARAMETERS
======================================================================

Trainable Parameters (should only be LoRA):
  encoder.layer.0.attention.self.lora_q.lora_A: torch.Size([8, 768]) (6,144 params)
  encoder.layer.0.attention.self.lora_q.lora_B: torch.Size([768, 8]) (6,144 params)
  encoder.layer.0.attention.self.lora_k.lora_A: torch.Size([8, 768]) (6,144 params)
  encoder.layer.0.attention.self.lora_k.lora_B: torch.Size([768, 8]) (6,144 params)
  encoder.layer.1.attention.self.lora_q.lora_A: torch.Size([8, 768]) (6,144 params)
  encoder.layer.1.attention.self.lora_q.lora_B: torch.Size([768, 8]) (6,144 params)
  encoder.layer.1.attention.self.lora_k.lora_A: torch.Size([8, 768]) (6,144 params)
  encoder.layer.1.attention.self.lora_k.lora_B: torch.Size([768, 8]) (6,144 params)
  encoder.layer.2.attention.self.lora_q.lora_A: torch.Size([8, 768]) (6,144 params)
  encoder.layer.2.attention.self.lora_q.lora_B: torch.Size([768, 8]) (6,144 params)
  encoder.layer.2.attention.self.lora_k.lora_A: torch.Size([8, 768]) (6,144 params)
  encoder.layer.2.attention.self.lora_k.lora_B: torch.Size([768, 8]) (6,144 params)
  encoder.layer.3.attention.self.lora_q.lora_A: torch.Size([8, 768]) (6,144 params)
  encoder.layer.3.attention.self.lora_q.lora_B: torch.Size([768, 8]) (6,144 params)
  encoder.layer.3.attention.self.lora_k.lora_A: torch.Size([8, 768]) (6,144 params)
  encoder.layer.3.attention.self.lora_k.lora_B: torch.Size([768, 8]) (6,144 params)
  encoder.layer.4.attention.self.lora_q.lora_A: torch.Size([8, 768]) (6,144 params)
  encoder.layer.4.attention.self.lora_q.lora_B: torch.Size([768, 8]) (6,144 params)
  encoder.layer.4.attention.self.lora_k.lora_A: torch.Size([8, 768]) (6,144 params)
  encoder.layer.4.attention.self.lora_k.lora_B: torch.Size([768, 8]) (6,144 params)
  encoder.layer.5.attention.self.lora_q.lora_A: torch.Size([8, 768]) (6,144 params)
  encoder.layer.5.attention.self.lora_q.lora_B: torch.Size([768, 8]) (6,144 params)
  encoder.layer.5.attention.self.lora_k.lora_A: torch.Size([8, 768]) (6,144 params)
  encoder.layer.5.attention.self.lora_k.lora_B: torch.Size([768, 8]) (6,144 params)
  encoder.layer.6.attention.self.lora_q.lora_A: torch.Size([8, 768]) (6,144 params)
  encoder.layer.6.attention.self.lora_q.lora_B: torch.Size([768, 8]) (6,144 params)
  encoder.layer.6.attention.self.lora_k.lora_A: torch.Size([8, 768]) (6,144 params)
  encoder.layer.6.attention.self.lora_k.lora_B: torch.Size([768, 8]) (6,144 params)
  encoder.layer.7.attention.self.lora_q.lora_A: torch.Size([8, 768]) (6,144 params)
```

TRAINING RESULTS

Shows the training configuration and successful loading of RoBERTa with gradient-free LoRA setup for **SST-2 dataset**

```
======================================================================
STARTING TRAINING
======================================================================
======================================================================
GRADIENT-FREE LoRA TRAINING
======================================================================

📊 Configuration:
  Dataset: sst2
  LoRA Rank: 8
  Intrinsic Dimension: 100
  Budget per layer: 400
  CMA-ES population: 8
  FWA population: 8
  Batch size: 8
  Initial Memory: 2343.85 MB

📦 Creating dataloaders...
  Train batches: 4
  Val batches: 4
  Test batches: 109

🤖 Loading RoBERTa with LoRA...
Loading weights: 100% ████████████████████████████████ 197/197 [00:00<00:00, 253.86it/s, Materializing param=encoder.layer.11.output.dense.weight]
RobertaModel LOAD REPORT from: roberta-base
```

Shows the mean and standard deviation of hidden states computed for all 12 layers, used to properly initialize the LoRA projection matrices.

```
📈 Computing hidden state statistics...
======================================================================
COMPUTING HIDDEN STATE STATISTICS
======================================================================

Layer 0: mean=0.020171, std=0.565223
Layer 1: mean=0.037757, std=0.791803
Layer 2: mean=0.038818, std=0.791111
Layer 3: mean=0.035548, std=0.786085
Layer 4: mean=0.034110, std=0.870647
Layer 5: mean=0.025639, std=0.875068
Layer 6: mean=0.029930, std=0.909863
Layer 7: mean=0.033934, std=0.910943
Layer 8: mean=0.038095, std=0.927999
Layer 9: mean=0.040177, std=0.925868
Layer 10: mean=0.034445, std=0.904333
Layer 11: mean=0.016740, std=0.430669

✓ Statistics computed for 12 layers
```

IMPLEMENTATION RESULTS – Module 4

Shows layer-wise CMA-ES optimization of intrinsic LoRA vectors, with loss decreasing per iteration until convergence for each vector.

```
Starting layer-wise optimization (12 layers)...

──────────────────────────────────────────────────────────────
ptimizing Layer 0 with CMA-ES
──────────────────────────────────────────────────────────────
 Iterations per vector: 12
 Population size: 8

 ============================================================
 Optimizing m1 (intrinsic vector 1/4)
 ============================================================
   Iter   1/12 | Best: 8.2666 | Avg: 8.2666 | Min: 8.2666
   Iter   5/12 | Best: 8.2666 | Avg: 8.2666 | Min: 8.2666
   Iter  10/12 | Best: 8.2666 | Avg: 8.2666 | Min: 8.2666

 ✓ m1 complete
   Final loss: 8.2666
   Time: 22.9s
```

```
 ============================================================
 Optimizing m2 (intrinsic vector 2/4)
 ============================================================
   Iter   1/12 | Best: 7.2136 | Avg: 9.8527 | Min: 7.2136
   Iter   5/12 | Best: 7.1296 | Avg: 7.8694 | Min: 7.1296
   Iter  10/12 | Best: 6.8477 | Avg: 7.4351 | Min: 6.8477

 ✓ m2 complete
   Final loss: 6.8477
   Time: 22.7s
```

```
============================================================
Optimizing m4 (intrinsic vector 4/4)
============================================================
  Iter   1/12 | Best: 7.7525 | Avg: 8.9723 | Min: 7.7525
  Iter   5/12 | Best: 7.6544 | Avg: 8.3695 | Min: 7.7841
  Iter  10/12 | Best: 7.6531 | Avg: 8.2059 | Min: 7.7638

✓ m4 complete
  Final loss: 7.6531
  Time: 22.4s
 ============================================================
```

Shows FWA-based optimization of Layer 11's intrinsic LoRA vectors (m1–m4), with loss gradually decreasing until convergence for each vector.

```
──────────────────────────────────────────────────────────────────────
Optimizing Layer 11 with FWA
──────────────────────────────────────────────────────────────────────
  Iterations per vector: 12
  Population size: 8

  ============================================================
  Optimizing m1 (intrinsic vector 1/4)
  ============================================================
    Iter   1/12 | Best: 6.7506 | Avg: 6.7506 | Min: 6.7506
    Iter   5/12 | Best: 6.7506 | Avg: 6.7506 | Min: 6.7506
    Iter  10/12 | Best: 6.7506 | Avg: 6.7506 | Min: 6.7506

  ✓ m1 complete
    Final loss: 6.7506
    Time: 84.4s

  ============================================================
  Optimizing m2 (intrinsic vector 2/4)
  ============================================================
    Iter   1/12 | Best: 6.7192 | Avg: 6.7667 | Min: 6.7192
    Iter   5/12 | Best: 6.7164 | Avg: 6.7171 | Min: 6.7164
    Iter  10/12 | Best: 6.7117 | Avg: 6.7128 | Min: 6.7117

  ✓ m2 complete
    Final loss: 6.7090
    Time: 84.0s

  ============================================================
  Optimizing m3 (intrinsic vector 3/4)
  ============================================================
    Iter   1/12 | Best: 6.7090 | Avg: 6.7090 | Min: 6.7090
    Iter   5/12 | Best: 6.7090 | Avg: 6.7090 | Min: 6.7090
    Iter  10/12 | Best: 6.7090 | Avg: 6.7090 | Min: 6.7090

  ✓ m3 complete
    Final loss: 6.7090
    Time: 84.3s

  ============================================================
  Optimizing m4 (intrinsic vector 4/4)
  ============================================================
    Iter   1/12 | Best: 6.6671 | Avg: 6.7064 | Min: 6.6671
    Iter   5/12 | Best: 6.6639 | Avg: 6.6653 | Min: 6.6639
    Iter  10/12 | Best: 6.6595 | Avg: 6.6607 | Min: 6.6595

  ✓ m4 complete
    Final loss: 6.6587
    Time: 84.4s
```

IMPLEMENTATION RESULTS – Module 5

Shows final test performance, training time, memory usage, and successful saving

```
  Layer 11 Summary:
    Optimizer: FWA
    Time: 336.99s
    Val Accuracy: 0.7778
    Val F1: 0.7729
    Memory: 2877.63 MB

======================================================================
FINAL EVALUATION ON TEST SET
======================================================================
  Verbalizer mapping: {10999: 0, 12338: 1}

📊 Test Results:
  Accuracy:  0.8122
  Precision: 0.8222
  Recall:    0.8122
  F1 Score:  0.8072

⏱️  Training Time: 2564.96s (42.75min)
💾 Peak Memory: 2877.63 MB
📈 Memory Increase: 533.78 MB

💾 Saving model to /content/drive/MyDrive/FYP_GradientFree_LoRA/trained_models/sst2_rank8_dim100...
✓ Model and results saved!

======================================================================
✓ TRAINING COMPLETE!
======================================================================
```

Shows the training configuration and successful loading of RoBERTa with gradient-free LoRA setup for **AGNews dataset**.

```
================================================================================
STARTING TRAINING
================================================================================
================================================================================
GRADIENT-FREE LoRA TRAINING
================================================================================

Configuration:
 Dataset: ag_news
 LoRA Rank: 8
 Intrinsic Dimension: 100
 Batch size: 8
 Initial Memory: 1869.39 MB

Creating dataloaders...
 Train batches: 8
 Val batches: 8
 Test batches: 950

Loading RoBERTa with LoRA...
```




```
RobertaModel LOAD REPORT from: roberta-base
```

Shows the mean and standard deviation of hidden states computed for all 12 layers, used to properly initialize the LoRA projection matrices.

```
📈 Computing hidden state statistics...
======================================================================
COMPUTING HIDDEN STATE STATISTICS
======================================================================
 Layer 0: mean=0.021458, std=0.567184
 Layer 1: mean=0.037411, std=0.819614
 Layer 2: mean=0.037971, std=0.799526
 Layer 3: mean=0.036904, std=0.788980
 Layer 4: mean=0.035899, std=0.870977
 Layer 5: mean=0.026837, std=0.882326
 Layer 6: mean=0.030411, std=0.916583
 Layer 7: mean=0.033845, std=0.922043
 Layer 8: mean=0.038162, std=0.939150
 Layer 9: mean=0.040463, std=0.945455
 Layer 10: mean=0.034104, std=0.909338
 Layer 11: mean=0.016173, std=0.410982

 ✓ Statistics computed for 12 layers
```

IMPLEMENTATION RESULTS – Module 4

Shows layer-wise CMA-ES optimization of intrinsic LoRA vectors, with loss decreasing per iteration until convergence for each vector.

```
──────────────────────────────────────────────────────────────
Optimizing Layer 0 with CMA-ES
──────────────────────────────────────────────────────────────
 Iterations per vector: 12
 Population size: 8

============================================================
 Optimizing m1 (intrinsic vector 1/4)
============================================================
 Iter   1/12 | Best: 3.4832 | Avg: 3.4832 | Min: 3.4832
 Iter   5/12 | Best: 3.4832 | Avg: 3.4832 | Min: 3.4832
 Iter  10/12 | Best: 3.4832 | Avg: 3.4832 | Min: 3.4832

 ✓ m1 complete
 Final loss: 3.4832
 Time: 36.1s
```

```
============================================================
 Optimizing m2 (intrinsic vector 2/4)
============================================================
 Iter   1/12 | Best: 4.3699 | Avg: 6.6152 | Min: 4.3699
 Iter   5/12 | Best: 3.2726 | Avg: 4.1559 | Min: 3.4628
 Iter  10/12 | Best: 3.1305 | Avg: 3.6630 | Min: 3.4389

 ✓ m2 complete
 Final loss: 3.1305
 Time: 36.9s
```

```
============================================================
 Optimizing m4 (intrinsic vector 4/4)
============================================================
 Iter   1/12 | Best: 4.4111 | Avg: 4.8004 | Min: 4.4111
 Iter   5/12 | Best: 4.1364 | Avg: 4.7285 | Min: 4.1364
 Iter  10/12 | Best: 3.9693 | Avg: 4.2214 | Min: 4.0680

 ✓ m4 complete
 Final loss: 3.8444
 Time: 37.7s
```

Shows FWA-based optimization of Layer 11's intrinsic LoRA vectors (m1–m4), with loss gradually decreasing until convergence for each vector.

```
Optimizing Layer 11 with FWA
──────────────────────────────────────────────────────────────

 Iterations per vector: 12
 Population size: 5

============================================================
 Optimizing m1 (intrinsic vector 1/4)
============================================================
 Iter   1/12 | Best: 6.3363 | Avg: 6.3363 | Min: 6.3363
 Iter   5/12 | Best: 6.3363 | Avg: 6.3363 | Min: 6.3363
 Iter  10/12 | Best: 6.3363 | Avg: 6.3363 | Min: 6.3363

 ✓ m1 complete
 Final loss: 6.3363
 Time: 75.3s

============================================================
 Optimizing m2 (intrinsic vector 2/4)
============================================================
 Iter   1/12 | Best: 6.3166 | Avg: 6.3572 | Min: 6.3166
 Iter   5/12 | Best: 6.3142 | Avg: 6.3151 | Min: 6.3142
 Iter  10/12 | Best: 6.3112 | Avg: 6.3121 | Min: 6.3112

 ✓ m2 complete
 Final loss: 6.3091
 Time: 75.7s

============================================================
 Optimizing m3 (intrinsic vector 3/4)
============================================================
 Iter   1/12 | Best: 6.3091 | Avg: 6.3091 | Min: 6.3091
 Iter   5/12 | Best: 6.3091 | Avg: 6.3091 | Min: 6.3091
 Iter  10/12 | Best: 6.3091 | Avg: 6.3091 | Min: 6.3091

 ✓ m3 complete
 Final loss: 6.3091
 Time: 75.4s

============================================================
 Optimizing m4 (intrinsic vector 4/4)
============================================================
 Iter   1/12 | Best: 6.3034 | Avg: 6.3195 | Min: 6.3034
 Iter   5/12 | Best: 6.3025 | Avg: 6.3028 | Min: 6.3025
 Iter  10/12 | Best: 6.3002 | Avg: 6.3006 | Min: 6.3002

 ✓ m4 complete
 Final loss: 6.2998
 Time: 75.1s
```

IMPLEMENTATION RESULTS – Module 5

Shows final test performance, training time, memory usage, and successful saving

```
 Layer 11 Summary:
 Optimizer: FWA
 Time: 301.43s
 Val Accuracy: 0.8000
 Val F1: 0.7874
 Memory: 2693.11 MB

======================================================================
FINAL EVALUATION ON TEST SET
======================================================================
  Verbalizer mapping: {10988: 0, 10554: 1, 18562: 2, 14396: 3}

Test Results:
 Accuracy: 0.8284
 Precision: 0.8384
 Recall: 0.8284
 F1 Score: 0.8234

Training Time: 2757.80s (45.96 min)
Peak Memory: 2693.11 MB
Memory Increase: 823.72 MB
✓ Model and results saved!

======================================================================
```

Performance Results of DFO-LoRA across various datasets

Method	Accuracy(%)	F1-Score(%)	Memory (MB)	Time (min)
Proposed LoRA with DFO (SST-2)	81.22	80.72	2878	42.75
Proposed LoRA with DFO (Yelp)	87.09	86.79	2419	37.41
Proposed LoRA with DFO (AGNews)	82.84	82.34	2693	45.96
Proposed LoRA with DFO (DBPedia)	88.75	88.55	2276	44.30

Comparison between Gradient-Based LoRA and Proposed Gradient-Free LoRA on SST-2 dataset

Method	Accuracy	F1-Score	Memory (MB)
LoRA with Adam	77.45%	76.02%	2983
Proposed LoRA with DFO	81.22%	80.72%	2878

Comparison Results on BBT and Proposed DFO-LoRA on SST-2 Dataset

Method	Accuracy(%)	F1-Score(%)	Time (min)	Memory Used (MB)
BBT on SST-2	50	32.31	63.83	3146
DFO-LoRA on SST-2	81.22	80.72	42.75	2878

Performance of Manual Prompting and In-Context Learning across Datasets

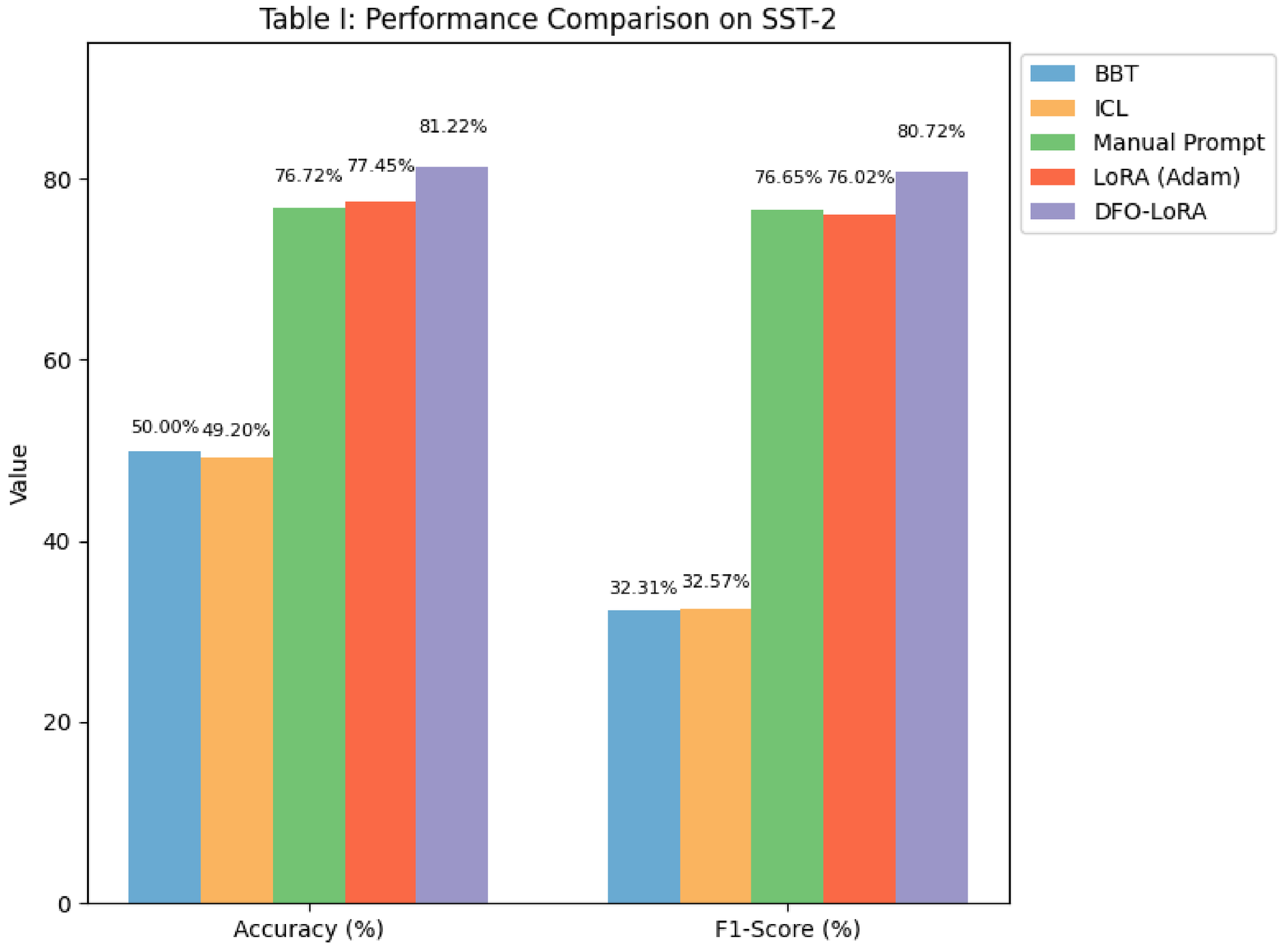
Accuracy

Method	SST-2	Yelp	AGNews	DBPedia
Manual Prompt	76.72	73.88	67.09	31.92
In-context learning	49.20	70.76	25.80	46.04

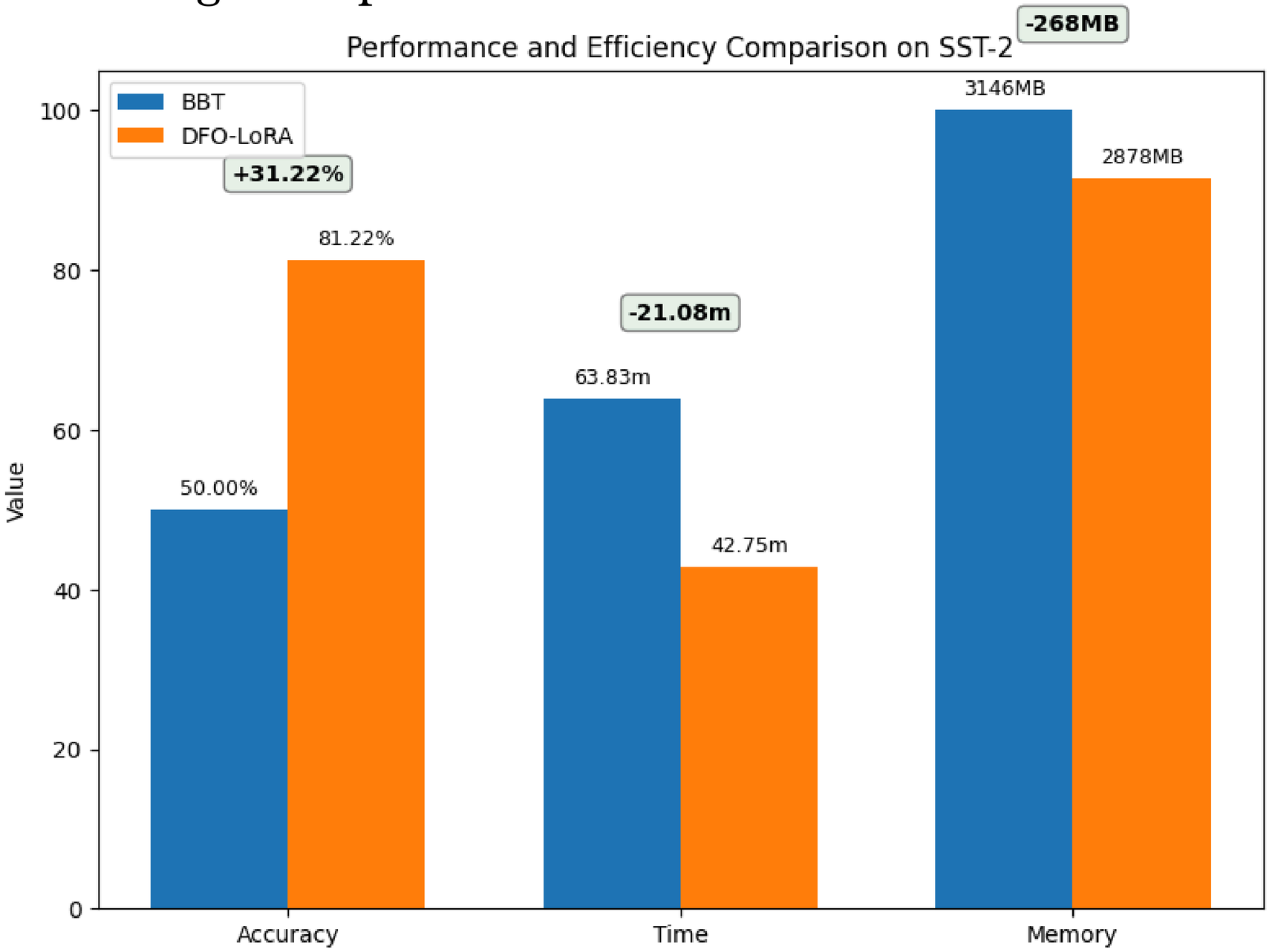
F-1 Score

Method	SST-2	Yelp	AGNews	DBPedia
Manual Prompt	76.65	72.66	65.51	29.72
In-context learning	32.57	69.58	11.65	42.24

Performance of all methods on SST-2, with DFO-LoRA performing best



DFO-LoRA improves accuracy while reducing time and memory usage compared to BBT on the SST-2 dataset.



Thank You